

# halfedge: OBJ / PLY / STL 读写与网格构建模块设计文档

src/io.rs

halfedge 库

2026-07-05

## 目录

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| <b>1</b> | <b>模块定位</b>                                    | <b>2</b>  |
| <b>2</b> | <b>OBJ 格式约定</b>                                | <b>2</b>  |
| <b>3</b> | <b>错误类型</b>                                    | <b>3</b>  |
| <b>4</b> | <b>build_mesh_from_vertices_and_faces 算法</b>   | <b>3</b>  |
| 4.1      | 整体流程                                           | 3         |
| 4.2      | 步骤 1-2: 顶点与内部半边                                | 4         |
| 4.3      | 步骤 3: twin 建立                                  | 4         |
| 4.4      | 步骤 4: 边界半边 next/prev 链接 (walking 算法)           | 4         |
| <b>5</b> | <b>面绕向一致性约束</b>                                | <b>5</b>  |
| <b>6</b> | <b>OBJ 序列化</b>                                 | <b>6</b>  |
| <b>7</b> | <b>STL 读写</b>                                  | <b>6</b>  |
| 7.1      | ASCII 格式                                       | 6         |
| 7.2      | 二进制格式                                          | 7         |
| 7.3      | ASCII / 二进制自动判别                                | 7         |
| 7.4      | STL 序列化                                        | 7         |
| 7.5      | StlError                                       | 8         |
| 7.6      | 顶点去重原理                                         | 8         |
| <b>8</b> | <b>统一入口: load_mesh / save_mesh / MeshError</b> | <b>8</b>  |
| 8.1      | MeshError 枚举                                   | 9         |
| 8.2      | 使用示例                                           | 9         |
| <b>9</b> | <b>测试覆盖</b>                                    | <b>10</b> |

## 1 模块定位

`io.rs` 提供五组能力:

| 类别    | API                                             | 语义                                                          |
|-------|-------------------------------------------------|-------------------------------------------------------------|
| 构建    | <code>build_mesh_from_vertices_and_faces</code> | 顶点 + 三角面索引 → 完整半边网格                                         |
|       | <code>build_mesh_from_polygons</code>           | 顶点 + 任意多边形面 → 三角化半边网格                                       |
| OBJ 读 | <code>load_obj(path)</code>                     | 从文件加载                                                       |
|       | <code>parse_obj(text)</code>                    | 从文本解析                                                       |
| OBJ 写 | <code>save_obj(mesh, path)</code>               | 保存到文件                                                       |
|       | <code>format_obj(mesh)</code>                   | 序列化为文本                                                      |
| PLY 读 | <code>load_ply(path)</code>                     | 从文件加载 PLY ASCII                                             |
|       | <code>parse_ply(text)</code>                    | 从文本解析 PLY ASCII                                             |
| PLY 写 | <code>save_ply(mesh, path)</code>               | 保存到 PLY 文件                                                  |
|       | <code>format_ply(mesh)</code>                   | 序列化为 PLY 文本                                                 |
| STL 读 | <code>load_stl(path)</code>                     | 从文件加载 (自动判别 ASCII / 二进制)                                    |
|       | <code>parse_stl_bytes(bytes)</code>             | 从字节切片解析 (启发式判别)                                             |
|       | <code>parse_stl_ascii(text)</code>              | 从文本解析 STL ASCII                                             |
|       | <code>parse_stl_binary(bytes, n)</code>         | 从字节解析 STL 二进制                                               |
| STL 写 | <code>save_stl_ascii(mesh, path)</code>         | 保存为 STL ASCII 文件                                            |
|       | <code>format_stl_ascii(mesh)</code>             | 序列化为 STL ASCII 文本                                           |
|       | <code>save_stl_binary(mesh, path)</code>        | 保存为 STL 二进制文件                                               |
|       | <code>format_stl_binary(mesh)</code>            | 序列化为 STL 二进制字节                                              |
| 统一入口  | <code>MeshError</code>                          | 统一错误枚举 ( <code>Obj / Ply / Stl / UnsupportedFormat</code> ) |
|       | <code>load_mesh(path)</code>                    | 按扩展名自动选择 OBJ / PLY / STL 解析器                                |
|       | <code>save_mesh(mesh, path)</code>              | 按扩展名自动选择 OBJ / PLY / STL 序列化器                               |

## 2 OBJ 格式约定

```
1 #          1-based
2 v 0.0 0.0 0.0
3 v 1.0 0.0 0.0
4 v 0.0 1.0 0.0
5 #          / n-gon          v / v/vt / v/vt/vn          v
6 f 1 2 3
```

```

7 f 1/1/1 2/2/1 3/3/1
8 f 1 2 3 4 # n-gon fan
9 # OBJ
10 f -3 -2 -1

```

- 仅解析 `v` 与 `f` 行；`vt`、`vn`、`g`、`usemtl` 等其他行被忽略；
- 顶点索引 1-based；负数表示从末尾倒数；
- 支持 `n-gon`：面顶点数  $\geq 3$  即可，自动按 `fan` 方式三角化；仅当面顶点数  $< 3$  时返回 `NotTriangular` 错误；
- 索引越界返回 `IndexOutOfRange` 错误。

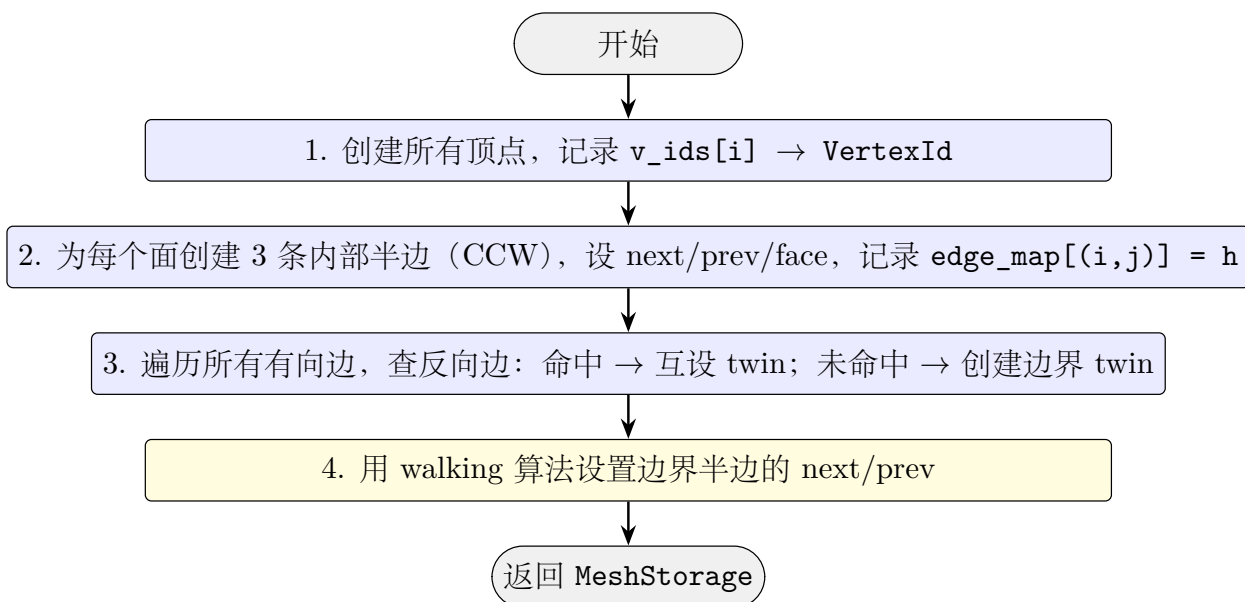
### 3 错误类型

`ObjError` 共 4 个变体：

| 变体                                                    | 触发场景          |
|-------------------------------------------------------|---------------|
| <code>Io(std::io::Error)</code>                       | 文件读写错误        |
| <code>Parse{line, msg}</code>                         | 解析失败（行号 + 描述） |
| <code>IndexOutOfRange{line, idx, vertex_count}</code> | 面索引越界         |
| <code>NotTriangular{line, face_verts}</code>          | 面顶点数 $< 3$    |

## 4 build\_mesh\_from\_vertices\_and\_faces 算法

### 4.1 整体流程



**容量预分配** 入口处调用 `mesh.reserve(vertices.len(), faces.len() * 6, faces.len())` 预分配内存, 其中半边容量上限 `faces.len() * 6` 来自每三角面 3 条内部半边 + 3 条潜在边界 twin。`build_mesh_from_polygons` 同样在入口处调用 `reserve`, 半边容量按  $\sum_i k_i \times 2$  计算 ( $k_i$  为第  $i$  个面的边数)。预分配可将批量构建的 rehash 次数降为 0。

## 4.2 步骤 1–2: 顶点与内部半边

对每个面  $[i_0, i_1, i_2]$  (0-based), 创建三条半边:

$$h_0 : v_{i_0} \rightarrow v_{i_1}, \quad h_1 : v_{i_1} \rightarrow v_{i_2}, \quad h_2 : v_{i_2} \rightarrow v_{i_0}.$$

设置  $\text{next}(h_0) = h_1$ ,  $\text{next}(h_1) = h_2$ ,  $\text{next}(h_2) = h_0$ , prev 反向,  $\text{face} = f$ 。记录 `edge_map`:  $(i_0, i_1) \rightarrow h_0$ ,  $(i_1, i_2) \rightarrow h_1$ ,  $(i_2, i_0) \rightarrow h_2$ 。

## 4.3 步骤 3: twin 建立

对每条有向边  $(a, b) \rightarrow h$ :

- 查 `edge_map` 是否有  $(b, a)$ :
  - 命中  $h'$ : 内部边, 互设  $h.\text{twin} = h'$ ,  $h'.\text{twin} = h$ ;
  - 未命中: 边界边, 创建反向边界半边  $h^*$  ( $\text{face} = \text{None}$ ), 互设 twin, 加入 `boundary_twins` 列表。

## 4.4 步骤 4: 边界半边 next/prev 链接 (walking 算法)

**问题:** 边界半边  $bh$  ( $A \rightarrow B$ ,  $\text{face} = \text{None}$ ) 需要设置 next 与 prev, 使其与其它边界半边构成外环。

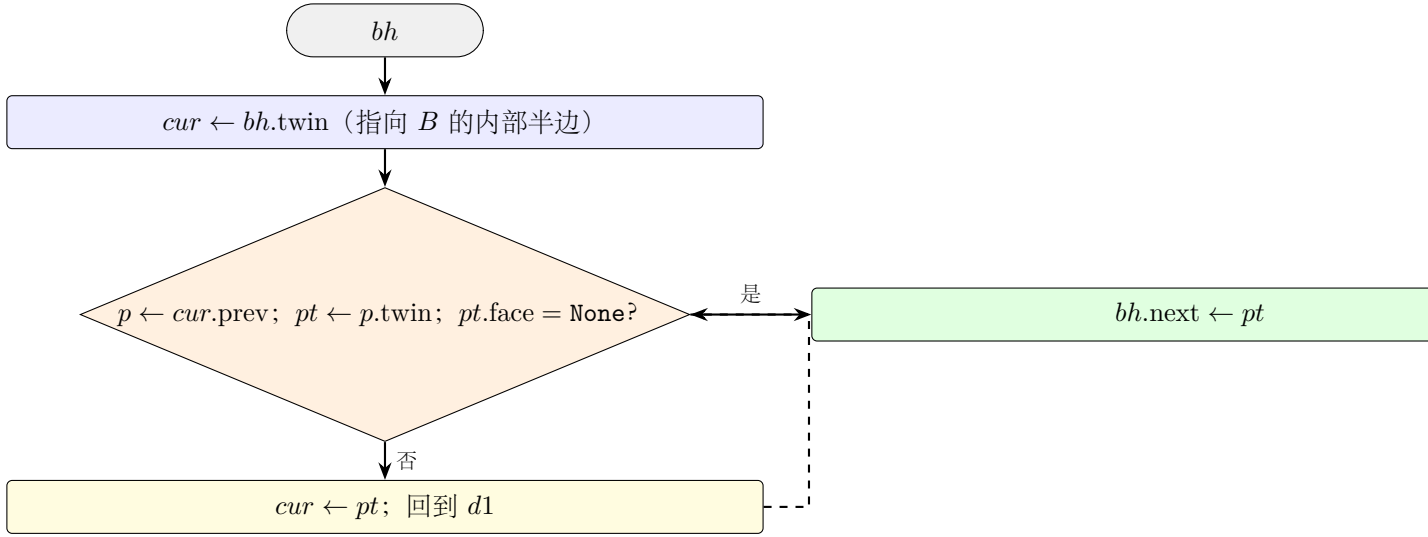
**旧公式 (仅适用单三角形):**

$$bh.\text{next} = bh.\text{twin}.\text{prev}.\text{twin}, \quad bh.\text{prev} = bh.\text{twin}.\text{next}.\text{twin}.$$

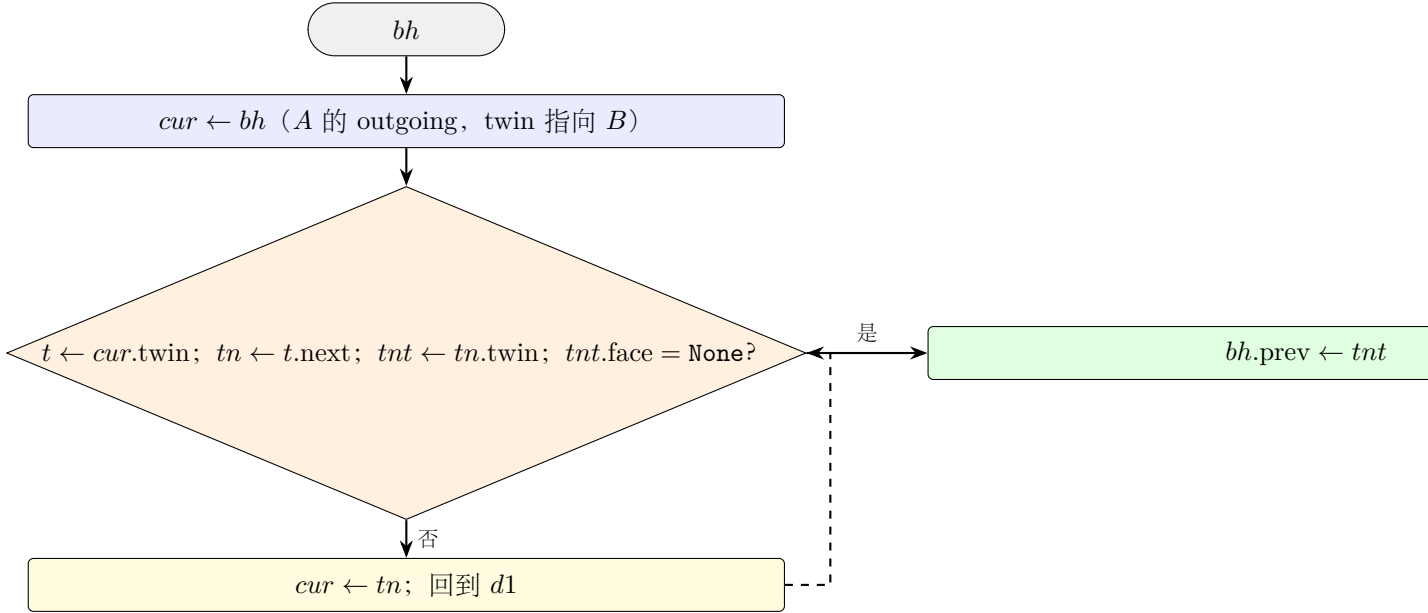
当  $bh.\text{twin}.\text{prev}$  的 twin 是内部半边时, 此公式失效。

**新算法 (walking):**

求  $bh.\text{next}$ : 绕  $bh.\text{tip}$  (即  $B$ ) 沿 CCW 方向旋转, 直到找到边界 outgoing 半边。



求  $bh.prev$ : 绕  $bh.origin$  (即  $A$ ) 沿 CW 方向旋转, 直到找到  $twin$  为边界半边的 outgoing。



**终止性:** 边界半边的 tip 必为边界顶点, 边界顶点的 outgoing 环是开链 (非闭合), walking 必终止。代码设  $\text{max\_iter} = \text{halfedge\_count} + 1$  作安全阀。

## 5 面绕向一致性约束

**build\_mesh\_from\_vertices\_and\_faces** 假设所有面绕向一致 (每条无向边在两个面中方向相反)。若输入违反此约束 (如两个面都包含有向边  $a \rightarrow b$ ) , 则:

1. **edge\_map** 第二次插入会覆盖第一次;
2. 第一次插入的半边  $h_{\text{first}}$  不在 **directed\_edges** 中, 永远不会被设置 **twin**;
3. 第二次插入的半边  $h_{\text{second}}$  查找  $(b, a)$  失败, 创建边界 **twin**;

4. 结果:  $h_{\text{first}}.\text{twin} = \text{None}$ , `validate_topology` 会报告 `HalfEdgeDanglingTwin` 或入口不一致。

**使用建议:**调用方应保证面绕向一致(如所有面 CCW 朝外);若不确定,应先用 `validate_topology` 检查构建结果。

## 6 OBJ 序列化

`format_obj` 输出格式:

- 顶点按 `vertex_ids()` 顺序, 分配 1-based 索引;
- 面按 `face_ids()` 顺序, 每行 `f i j k`;
- 非三角面 (边界环长度  $\neq 3$ ) 跳过;
- 顶点位置用 `:.6` 格式化 (6 位小数)。

## 7 STL 读写

STL (STereoLithography) 格式广泛用于 3D 打印与 CAD 交换。`io.rs` 支持 ASCII 与二进制两种变体, 并提供自动判别。

### 7.1 ASCII 格式

```
1 solid model
2   facet normal 0.0 0.0 1.0
3     outer loop
4       vertex 0.0 0.0 0.0
5       vertex 1.0 0.0 0.0
6       vertex 0.0 1.0 0.0
7     endloop
8   endfacet
9 endsolid model
```

`parse_stl_ascii` 解析流程:

1. 按行拆分, 跳过 `solid` / `endsolid` / `facet normal` / `outer loop` / `endloop` / `endfacet` 等关键字;
2. 提取所有 `vertex x y z` 行为顶点位置;
3. 每连续 3 个顶点构成一个三角面;
4. **顶点去重:**用 `f64::to_bits()` 的位模式作为 `HashMap` 键, 相同位置的顶点复用同一 `VertexId`;
5. 调用 `build_mesh_from_vertices_and_faces` 构建半边网格。

## 7.2 二进制格式

```
1 [80          ]
2 [4      little-endian      N]
3 [50      × N      12      + 36      + 2      ]
```

`parse_stl_binary` 解析流程:

1. 跳过 80 字节文件头;
2. 读取 4 字节 `u32` 作为面数  $N$ ;
3. 校验文件长度  $= 84 + 50N$ , 不匹配返回 `BadBinarySize` 错误;
4. 每个三角面读取 50 字节, 提取三顶点位置 (法向被忽略, 由网格重建);
5. 顶点去重: 用 `f32::to_bits()` 的位模式 (`u32` 数组) 作为 `HashMap` 键, 相同位置的顶点复用同一 `VertexId`;
6. 调用 `build_mesh_from_vertices_and_faces` 构建半边网格。

## 7.3 ASCII / 二进制自动判别

`parse_stl_bytes(bytes)` 使用启发式判别:

- 检查文件长度: 若  $\text{len} = 84 + 50N$  ( $N$  为前 80 字节后的 4 字节 `u32`), 判定为二进制;
- 否则尝试以 UTF-8 解码后调用 `parse_stl_ascii`;
- 两者都失败返回 `StlError::Parse`。

为何用文件长度判别? ASCII STL 起始 80 字节通常含 `solid` 关键字, 但二进制文件头也可能是任意 80 字节。文件长度恰好等于  $84 + 50N$  是二进制的强信号。

## 7.4 STL 序列化

`format_stl_ascii` 输出标准 ASCII 格式:

- `solid halfedge` 起始;
- 每面输出 `facet normal` (用 `face_normal` 计算) + 三顶点 `vertex`;
- `endsolid halfedge` 结束。

`format_stl_binary` 输出标准二进制格式:

- 80 字节文件头 (前缀 `halfedge binary stl`, 余 0);
- 4 字节 little-endian 面数  $N$ ;
- 每面 50 字节: 法向 (12B) + 三顶点 (36B) + 属性 (2B, 全 0)。

## 7.5 StlError

| 变体                                           | 触发场景                |
|----------------------------------------------|---------------------|
| <code>Io(std::io::Error)</code>              | 文件读写错误              |
| <code>Parse{line, msg}</code>                | ASCII 解析失败（行号 + 描述） |
| <code>BadBinarySize{actual, expected}</code> | 二进制文件长度不匹配          |
| <code>NotTriangular{face_verts}</code>       | 非三角面（顶点数 $\neq 3$ ） |

## 7.6 顶点去重原理

STL 文件每个三角面独立存储三顶点，无共享顶点索引。直接构建网格会产生  $3F$  个 `VertexId`（ $F$  为面数），而非实际的  $V$  个顶点（ $V \approx F/2$ ）。

去重算法：

$$\text{key}(p) = [\text{f64}::\text{to\_bits}(p_x), \text{f64}::\text{to\_bits}(p_y), \text{f64}::\text{to\_bits}(p_z)]$$

- 用 `HashMap<[u64; 3], u32>`（ASCII）或 `HashMap<[u32; 3], u32>`（二进制，因 `f32`）；
- `to_bits()` 是位精确比较，浮点数  $0.0 \neq -0.0$ ；
- 命中已有索引则复用，否则分配新 `VertexId`。

为何用位精确而非  $\epsilon$ -tolerance？

- STL 文件中相同顶点的浮点表示通常**完全相同**（同一写入器输出）；
- 位精确比较  $O(1)$  且无歧义；
- 浮点容差比较需  $O(\log V)$  查找（KD-tree），且阈值敏感。

若 STL 文件含微小浮点误差导致同顶点不同表示，应先用 `weld_vertices`（按距离阈值合并）做后处理。

## 8 统一入口：load\_mesh / save\_mesh / MeshError

为简化上层调用，`io.rs` 提供按扩展名自动分派的统一接口：

| 扩展名               | load_mesh 调用                                        | save_mesh 调用                 |
|-------------------|-----------------------------------------------------|------------------------------|
| <code>.obj</code> | <code>load_obj</code>                               | <code>save_obj</code>        |
| <code>.ply</code> | <code>load_ply</code>                               | <code>save_ply</code>        |
| <code>.stl</code> | <code>load_stl</code>                               | <code>save_stl_binary</code> |
| 其他                | <code>Err(MeshError::UnsupportedFormat(ext))</code> |                              |



## 8.1 MeshError 枚举

MeshError 是 OBJ / PLY / STL 错误的**统一包装**，通过 From<ObjError>、From<PlyError> 与 From<StlError> 实现自动转换，便于在 ? 运算符中混用：

| 变体                        | 来源                       |
|---------------------------|--------------------------|
| Obj(ObjError)             | load_obj / save_obj 错误   |
| Ply(PlyError)             | load_ply / save_ply 错误   |
| Stl(StlError)             | load_stl / save_stl_* 错误 |
| UnsupportedFormat(String) | 未知扩展名                    |

## 8.2 使用示例

```
1 //
2 let mesh = halfedge::load_mesh("model.obj");
3 let mesh = halfedge::load_mesh("model.ply");
4
5 //
6 halfedge::save_mesh(&mesh, "out.ply");
7 halfedge::save_mesh(&mesh, "out.obj");
```

## 9 测试覆盖

| 测试名                                      | 验证点                                         |
|------------------------------------------|---------------------------------------------|
| 构建器                                      |                                             |
| build_mesh_basic_quad                    | 4 顶点 2 面 10 半边                              |
| build_mesh_passes_full_validation        | 构建结果通过完整校验                                  |
| build_mesh_closed_tetrahedron            | 闭合四面体 12 半边                                 |
| OBJ 往返与解析                                |                                             |
| obj_roundtrip_quad                       | 四边形 OBJ 往返一致                                |
| obj_roundtrip_tetrahedron                | 四面体 OBJ 往返一致                                |
| obj_parse_skips_comments_and_other_lines | 忽略 # / vt / vn / g 等                        |
| obj_parse_supports_v_vt_vn_format        | 解析 v/vt/vn 格式                               |
| obj_parse_negative_indices               | 负索引支持                                       |
| obj_parse_quadrilateral_face_succeeds    | 四边形面解析成功（支持 n-gon）                          |
| obj_parse_out_of_range_index_fails       | 越界索引报 <code>IndexOutOfRangeException</code> |
| obj_save_load_file_roundtrip             | 文件 IO 往返一致                                  |
| STL 往返与解析                                |                                             |
| stl_ascii_roundtrip                      | STL ASCII 往返一致                              |
| stl_binary_roundtrip                     | STL 二进制往返一致                                 |
| stl_ascii_parses_minimum_solid           | 最小 solid 块解析成功                              |
| stl_ascii_rejects_non_triangular         | 非三角面报 <code>NotTriangular</code>            |
| stl_binary_detects_size_mismatch         | 二进制长度不匹配报 <code>BadBinarySize</code>        |
| stl_file_roundtrip_ascii                 | STL ASCII 文件 IO 往返                          |
| stl_file_roundtrip_binary                | STL 二进制文件 IO 往返                             |
| 统一入口                                     |                                             |
| auto_detect_obj                          | <code>load_mesh(".obj")</code> 走 OBJ 路径     |
| auto_detect_ply                          | <code>load_mesh(".ply")</code> 走 PLY 路径     |
| auto_detect_stl                          | <code>load_mesh(".stl")</code> 走 STL 路径     |
| auto_detect_unsupported                  | 未知扩展名返回 <code>UnsupportedFormat</code>      |

全模块共 **22** 个单元测试，全库共 **371** 个。